

CHỦ ĐỀ: CODE SMELLS

ĐỀ: Giới thiệu các lỗi code smells của các lỗi code. Dấu hiệu nhận biết từng loại. Tìm một ví dụ ngắn gọn về lỗi và hướng refactor của nó.

Trước hết, ta sẽ chia code smell thành 5 nhóm phân loại như sau:

1. BLOATERS:

1.1. BLOATERS - LONG METHOD: a method contains many lines of codes , especially more than 10 lines of ones.

VD:

```
public double totalFee(){
    double totalTaxAfterExtra = tax + Extra;
    switch (season) {
        case "spring": return totalTaxAfterExtra *1.1;
        case "summer": return totalTaxAfterExtra *1.2;
        case "autumn": return totalTaxAfterExtra *1.8;
        case "winter": return totalTaxAfterExtra *3.0;
        default: System.out.println("Not valid season. You should enter only spring,summer,autumn or winter please.");
    }
    return 0.0;
}
```

To reduce the length of a method, we use **Extract Method**.

```
// SOLUTION: EXTRACT METHOD
public double totalTaxAfterExtra() {
    return tax + Extra;
}
public double totalFee(){
    // double totalTaxAfterExtra = tax + Extra;
    switch (season) {
        case "spring": return totalTaxAfterExtra() *1.1;
        case "summer": return totalTaxAfterExtra() *1.2;
        case "autumn": return totalTaxAfterExtra() *1.8;
        case "winter": return totalTaxAfterExtra() *3.0;
        default: System.out.println("Not valid season. You should enter only spring,summer,autumn or winter please.");
    }
    return 0.0;
}
```

1.2. BLOATERS - LARGE CLASS: A class contains many lines of code, too many methods /fields.

To refactor this problem, we can use

Extract class : Khi các behaviors of the large class can be spun off into a separate component.

```
public class Student {
    private String name;
    private String mssv;
    private String date;

    public Student(String name, String mssv, String date){
        this.name = name;
        this.mssv = mssv;
        this.date = date;
    }
    public void loginStudentAccount(){
        // TO DO
    }

    public void logOutStudentAccount(){
        // TO DO
    }

    public void checkValidDate(){
        // TODO
    }
    public void enrollCourse(){
        //To do
    }
    public void evaluateDegree(){
        //TODO
    }
    public void evaluateGPA(){
        //TODO
    }
    public void findStudentwithHighestGrade(){
        //TODO
    }
}
```

```

public class AccountStudent{
    private String name;
    private String mssv;
    private String date;
    public AccountStudent(String name,String mssv,String date){
        this.name = name;
        this.mssv = mssv;
        this.date= date;
    }
    public void loginStudentAccount(){
        // TO DO
    }
    public void logOutStudentAccount(){
        // TO DO
    }
    public void enrollCourse(){
        //To do
    }
    public String getName(){
        return name;
    }
    public String getMSSV(){
        return mssv;
    }
    public String getDate(){
        return date;
    }
}

public class toolExtra{
    public void checkValidDate(){
        // TODO
    }
    public void evaluateDegree(){
        //TODO
    }
    public void evaluateGPA(){
        //TODO
    }
    public void findStudentwwithHighestGrade(){
        //TODO
    }
}

public class Student {
    private AccountStudent account;
    private toolExtra tool;

    public Student(AccountStudent account, toolExtra tool){
        this.account = account;
        this.tool = tool;
    }
    public void learnCourse(){ // TODO }
}

```

Extract subclass: Khi các behavior of the large class được hiện thực và chỉ đc sử dụng trong một số trường hợp nhất định.

We will use polymorphism to solve such a large class.

```

class Employee {
    private String name;
    private String type; // "FULL_TIME", "PART_TIME", "INTERN"
    private double salary;
    private int workingHours;
    private int overTimeHours;

    public Employee(String name, String type, double salary, int workingHours, int overTimeHours) {
        this.name = name;
        this.type = type;
        this.salary = salary;
        this.workingHours = workingHours;
        this.overTimeHours = overTimeHours;
    }

    public double calculateSalary() {
        if (type.equals("FULL_TIME")) {
            return salary + overTimeHours * 100000;
        } else if (type.equals("PART_TIME")) {
            return workingHours * 50000;
        } else if (type.equals("INTERN")) {
            return salary * 0.5;
        }
        return 0;
    }

    public String getJobDescription() {
        if (type.equals("FULL_TIME")) {
            return "Nhân viên chính thức, có phụ cấp tăng ca";
        } else if (type.equals("PART_TIME")) {
            return "Nhân viên bán thời gian, tính lương theo giờ";
        } else if (type.equals("INTERN")) {
            return "Thực tập sinh, hỗ trợ dự án";
        }
        return "Không xác định";
    }

    public void work() {
        if (type.equals("FULL_TIME")) {
            System.out.println(name + " đang làm full-time");
        } else if (type.equals("PART_TIME")) {
            System.out.println(name + " đang làm part-time");
        } else if (type.equals("INTERN")) {
            System.out.println(name + " đang học việc");
        }
    }
}

class FullTimeEmployee extends Employee {
    private double baseSalary;
    private int overTimeHours;

    public FullTimeEmployee(String name, double baseSalary, int overTimeHours) {
        super(name);
        this.baseSalary = baseSalary;
        this.overTimeHours = overTimeHours;
    }

    @Override
    public double calculateSalary() {
        return baseSalary + overTimeHours * 100000;
    }

    @Override
    public String getJobDescription() {
        return "Nhân viên chính thức, có phụ cấp tăng ca";
    }

    @Override
    public void work() {
        System.out.println(name + " đang làm full-time");
    }
}

class PartTimeEmployee extends Employee {
    private int workingHours;

    public PartTimeEmployee(String name, int workingHours) {
        super(name);
        this.workingHours = workingHours;
    }

    @Override
    public double calculateSalary() {
        return workingHours * 50000;
    }

    @Override
    public String getJobDescription() {
        return "Nhân viên bán thời gian, tính lương theo giờ";
    }

    @Override
    public void work() {
        System.out.println(name + " đang làm part-time");
    }
}

class Intern extends Employee {
    private double allowance;

    public Intern(String name, double allowance) {
        super(name);
        this.allowance = allowance;
    }

    @Override
    public double calculateSalary() {
        return allowance * 0.5;
    }

    @Override
    public String getJobDescription() {
        return "Thực tập sinh, hỗ trợ dự án";
    }

    @Override
    public void work() {
        System.out.println(name + " đang học việc");
    }
}

```

Extract interface : dùng khi class làm quá nhiều thao tác nghiệp vụ, chúng ta có thể nhóm lại các thao tác thành các interface để sử dụng nó.

1.3. BLOATERS - LONG PARAMETER LIST: một phương thức chứa quá nhiều tham số truyền vào làm client có thể dễ bị confused về cách gọi tham số để truyền vào.

```
public class ExtraTool
{
    public double calculateMahattaDistance(double x, double y, double z){
        return Math.sqrt(x*x+y*y+z*z);
    }
}
```

To solve this problem , we can group involved variables into a grouped object.

```
public class Point {
    final private double x;
    final private double y;
    final private double z;
    public Point (double x, double y, double z){
        this.x =x;
        this.y =y;
        this.z =z;
    }
    public double getX(){return x;}
    public double getY(){return y;}
    public double getZ(){return z;}
}

public class ExtraTool
{
    public double calculateMahattaDistance(Point point){
        return Math.sqrt(point.getX()*point.getX()+ point.getY()*point.getY()+ point.getZ()*point.getZ());
    }
}
```

1.4. BLOATERS - PRIMITIVE OBSESSION: Use of primitives instead of small objects for simple tasks (such as currency, ranges, special strings for phone numbers, etc.). To refactor this, we should create the object to display type of data.

1.5. BLOATERS - DATA CLUMPS: Một nhóm biến/field/parameter thường xuyên xuất hiện cùng nhau nhưng chưa đc mô hình hóa để nhóm nó và biểu diễn nó như 1 nhóm kiểu dữ liệu chung.

```
public class Customer{
    private String name;
    private String address;
    private String City;
    private String postCode;
    private String phoneNumber;
    private String CCCD;

    public Customer( String name, String address, String City, String postCode, String phoneNumber, String CCCD){
        this.name= name;
        this.address= address;
        this.City= City;
        this.postCode= postCode;
        this.phoneNumber= phoneNumber;
        this.CCCD = CCCD;
    }
}
```

Vd:

To refactor this, we group all involved variables.

```

public class CustomerInfo{
    private String name;
    private String address;
    private String City;
    private String postCode;
    private String phoneNumber;
    private String CCCD;
    public CustomerInfo( String name,String address, String City,String postCode,String phoneNumber,String CCCD){
        this.name= name;
        this.address= address;
        this.City= City;
        this.postCode= postCode;
        this.phoneNumber= phoneNumber;
        this.CCCD = CCCD;
    }
}
public class Customer{
    private CustomerInfo info ;
    public Customer(CustomerInfo info ){
        this.info = info ;
    }
    public void postGood(){//TODO
}
}

```

2. OBJECT-ORIENTATION ABUSERS:

2.1. Switch statement: Instead of checking a type of class, we can use polymorphism to solve this.

```

class Employee {
    private String name;
    private String type; // "FULL_TIME", "PART_TIME", "INTERN"
    private double salary;
    private int workingHours;
    private int overTimeHours;

    public Employee(String name, String type, double salary, int workingHours, int overTimeHours) {
        this.name = name;
        this.type = type;
        this.salary = salary;
        this.workingHours = workingHours;
        this.overTimeHours = overTimeHours;
    }

    public double calculateSalary() {
        if (type.equals("FULL_TIME")) {
            return salary + overTimeHours * 10000;
        } else if (type.equals("PART_TIME")) {
            return workingHours * 5000;
        } else if (type.equals("INTERN")) {
            return salary * 0.5;
        }
        return 0;
    }

    public String getDescription() {
        if (type.equals("FULL_TIME")) {
            return "Nhân viên chính thức, có phụ cấp tăng ca";
        } else if (type.equals("PART_TIME")) {
            return "Nhân viên bán thời gian, tính lương theo giờ";
        } else if (type.equals("INTERN")) {
            return "Thực tập sinh, hỗ trợ dự án";
        }
        return "Không xác định";
    }

    public void work() {
        if (type.equals("FULL_TIME")) {
            System.out.println(name + " đang làm full-time");
        } else if (type.equals("PART_TIME")) {
            System.out.println(name + " đang làm part-time");
        } else if (type.equals("INTERN")) {
            System.out.println(name + " đang học việc");
        }
    }
}

// We will use polymorphism to solve such a large class.

public abstract class Employee {
    private double baseSalary;
    private int overTimeHours;

    public Employee(String name, double baseSalary, int overTimeHours) {
        super(name);
        this.baseSalary = baseSalary;
        this.overTimeHours = overTimeHours;
    }

    @Override
    public double calculateSalary() {
        return baseSalary + overTimeHours * 10000;
    }

    @Override
    public String getDescription() {
        return "Nhân viên chính thức, có phụ cấp tăng ca";
    }

    @Override
    public void work() {
        System.out.println(name + " đang làm full-time");
    }
}

class FullTimeEmployee extends Employee {
    public FullTimeEmployee(String name, double baseSalary, int overTimeHours) {
        super(name, baseSalary, overTimeHours);
    }

    @Override
    public double calculateSalary() {
        return baseSalary + overTimeHours * 10000;
    }

    @Override
    public String getDescription() {
        return "Nhân viên chính thức, có phụ cấp tăng ca";
    }

    @Override
    public void work() {
        System.out.println(name + " đang làm full-time");
    }
}

class PartTimeEmployee extends Employee {
    private int workingHours;

    public PartTimeEmployee(String name, int workingHours) {
        super(name, 0, 0);
        this.workingHours = workingHours;
    }

    @Override
    public double calculateSalary() {
        return workingHours * 5000;
    }

    @Override
    public String getDescription() {
        return "Nhân viên bán thời gian, tính lương theo giờ";
    }

    @Override
    public void work() {
        System.out.println(name + " đang làm part-time");
    }
}

class InternEmployee extends Employee {
    private double allowance;

    public InternEmployee(String name, double allowance) {
        super(name);
        this.allowance = allowance;
    }

    @Override
    public double calculateSalary() {
        return allowance * 0.5;
    }

    @Override
    public String getDescription() {
        return "Thực tập sinh, hỗ trợ dự án";
    }

    @Override
    public void work() {
        System.out.println(name + " đang học việc");
    }
}

```

2.2 .ALTERNATIVE CLASSES WITH DIFFERENT INTERFACES: Two classes perform identical functions but have different method names. To refactor this, we could create an interface for such classes.

```

public interface Animal {
    void say();
}

public class Dog implements Animal{
    @Override
    public void say(){
        System.out.println("Bark Bark");
    }
}

public class Cat implements Animal{
    @Override
    public void say(){
        System.out.println("Meoww Meoww");
    }
}

```

2.3. Temporary field: một class có field, nhưng field đó chỉ được dùng tạm thời trong một số method hoặc một số flow nhất định.

Ví dụ: Do `disCountRate` và `isHolidaySeason` chỉ được sử dụng trong `calculateFinalPrice` nên chỉ nên dùng local variable để xử lý bài toán này.

```
public class Order {
    private String customerName;
    private double totalAmount;

    private double discountRate;
    private boolean isHolidaySeason;

    public Order(String customerName, double totalAmount) {
        this.customerName = customerName;
        this.totalAmount = totalAmount;
    }

    public double calculateFinalPrice(boolean holiday) {
        this.isHolidaySeason = holiday;

        if (isHolidaySeason) {
            discountRate = 0.2;
        } else {
            discountRate = 0.0;
        }

        return totalAmount - totalAmount * discountRate;
    }
}
```

```
class Order {
    private String customerName;
    private double totalAmount;

    public Order(String customerName, double totalAmount) {
        this.customerName = customerName;
        this.totalAmount = totalAmount;
    }

    public double calculateFinalPrice(boolean holiday) {
        double discountRate = holiday ? 0.2 : 0.0;
        return totalAmount - totalAmount * discountRate;
    }
}
```

2.4. Refused Bequest: Subclass kế thừa từ superclass, nhưng không muốn hoặc không nên dùng một phần đáng kể của thứ nó được kế thừa.

```
public class Bird {
    public void fly() {
        System.out.println("Flying...");
    }
}

public class Penguin extends Bird {
    @Override
    public void fly(){
        throw new UnsupportedOperationException("Penguins can't fly");
    }
}
```

3. CHANGE PREVENTERS:

3.1. Divergent Change: Nhiều hướng thay đổi khác nhau đang đổ vào cùng một class, nếu như vậy chúng tỏ đây là 1 thiết kế rất tệ.

```
public class Employee {
    private String name;
    private double salary;

    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public double getSalary() {
        return salary;
    }

    public String getName() {
        return name;
    }
}

public class SalaryCalculator {
    public double calculate(Employee employee) {
        return employee.getSalary() * 1.1;
    }
}

public class EmployeeReportGenerator {
    public String generate(Employee employee) {
        return "Employee: " + employee.getName() + ", Salary: " + employee.getSalary();
    }
}
```

3.2. Parallel Inheritance Hierarchies: Bất kì khi nào bạn tạo ra 1 subclass for a class, cần tìm a subclass for another class.

3.3. Shotgun Surgery : tạo những sự thay đổi yêu cầu rằng bạn cần phải thay đổi những thay đổi nhỏ đối với những lớp khác.

- To refactor this, chúng ta cần phải gom lại những thay đổi lại thành 1 nơi để dễ dàng thay đổi logic khi cần thiết và dễ bảo trì sau này.

4. **DISPENSABLE:** Gồm các phần dư, không cần thiết hoặc tồn tại chỉ làm code khó đọc và khó sửa hơn.

4.1. Comment: We should comment only when we can't understand based on the meanings of codes.

4.2. Data class: Một class gần như chỉ chứa data(field +getter/setter), nhưng không có behaviour đúng nghĩa. Một class đúng nghĩa nên chứa các attribute và behavior method đi kèm với nhau.

```

public class Employee {
    private String name;
    private double salary;

    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public double getSalary() {
        return salary;
    }

    public String getName() {
        return name;
    }
}

```

4.3 Lazy Code: Một class cần phải thực sự thực hiện một chức năng thực sự cần thiết mới được giữ lại. Nếu không, nó nên được xóa đi.

4.4. Duplicate Code: Cùng một logic, cùng một logic nhưng lại bị lặp lại ở rất nhiều nơi. To refactor, use extract method for both classes.

4.5. Dead code: a variable, parameter, field, method or class is no longer used , bởi vì nó bị obsolete.

4.6. Speculative generality: Bạn đã tạo abstraction/hook/interface/class/hierarchy cho một chu cầu tương lai, .. nhưng chưa cần tới thậm chí không nhất định phải dùng đến nữa.

```

abstract class AnimalFactory {
    public abstract Animal createAnimal();
}

```

```

class DogFactory extends AnimalFactory {
    public Animal createAnimal() {
        return new Dog();
    }
}

```

5. **Couplers:** Các class đang phụ thuộc vào nhau quá nhiều.

5.1. FEATURE ENVY: Một method trong class A lại dùng data/method của class B nhiều hơn chính A.

. **To refactor:** chúng ta có thể dùng move method, extract method hoặc Extract class.

Ví dụ : Để xem membership có phải là VIP không, thì nên đánh giá nó trong Customer chỉ ra membership của nó.

```

public class Customer {
    private String name;
    private String membership;

    public Customer(String name, String membership) {
        this.name = name;
        this.membership = membership;
    }

    public String getMembership() {
        return membership;
    }
}

public class Order {
    private Customer customer;
    private double total;

    public Order(Customer customer, double total) {
        this.customer = customer;
        this.total = total;
    }

    public double calculateDiscount() {
        if (customer.getMembership().equals("VIP")) {
            return total * 0.2;
        }
        return total * 0.05;
    }
}

```

```

public class Customer {
    private String name;
    private String membership;

    public Customer(String name, String membership) {
        this.name = name;
        this.membership = membership;
    }

    public double getDiscountRate() {
        return membership.equals("VIP") ? 0.2 : 0.05;
    }
}

public class Order {
    private Customer customer;
    private double total;

    public Order(Customer customer, double total) {
        this.customer = customer;
        this.total = total;
    }

    public double calculateDiscount() {
        return total * customer.getDiscountRate();
    }
}

```

5.2. Inappropriate intimacy: một class sử dụng trường bên trong và methods of another class.

Ví dụ: ExtraTool đang can thiệp quá sâu vào trong Point cần phải refactor bằng cách move Method.

```

public class Point {
    final private double y;
    final private double z;
    public Point (double x, double y, double z){
        this.x =x;
        this.y =y;
        this.z =z;
    }
}

public class ExtraTool
{
    private Point X;
    public ExtraTool(Point X){
        this.X= X;
    }
    // public double calculateMahattaDistance(Point point){
    //     return Math.sqrt(point.getX()*point.getX()+ point.getY()*point.getY()+ point.getZ()*point.getZ());
    // }
    public double getX(){return X.x;}
    public double getY(){return X.y;}
    public double getZ(){return X.z;}
    public double calculateMahattaDistance(Point point){
        return Math.sqrt(getX()*getX()+ getY()*getY()+ getZ()*getZ());
    }
}

```

```

public class Point {
    final private double x;
    final private double y;
    final private double z;
    public Point (double x, double y, double z){
        this.x = x;
        this.y = y;
        this.z = z;
    }
    public double getX(){return x;}
    public double getY(){return y;}
    public double getZ(){return z;}
}

public class ExtraTool
{
    private Point X;
    public ExtraTool(Point X){
        this.X = X;
    }
    public double calculateMahattaDistance(Point point){
        return Math.sqrt(point.getX()*point.getX()+ point.getY()*point.getY()+ point.getZ()*point.getZ());
    }
}

```

5.3. Message chains : In code you see a series of calls resembling. Gọi một chuỗi các kênh gọi hàm $A() \Rightarrow B() \Rightarrow C() \Rightarrow D()$

Ví dụ: `order.getCustomer().getAddress().getCity().getName();`

5.4. Middleman : Nếu một class cần thực hiện thì phải gọi lại một class khác , giống như một class trung gian á. Đánh giá xem nó có thực sự cần thiết hay không? Nếu không, ta cần refactor nó một cách hợp lý, dùng phương pháp *remove Middle Man*.

Ví dụ: Để gọi `getDiscountRate ()` ta cần phải gọi như sau :

`order.getCustomer().getDiscountRate();`

```

public class Customer {
    private String name;
    private String membership;

    public Customer(String name, String membership) {
        this.name = name;
        this.membership = membership;
    }

    public double getDiscountRate() {
        return membership.equals("VIP") ? 0.2 : 0.05;
    }
}

public class Order {
    private Customer customer;
    private double total;

    public Order(Customer customer, double total) {
        this.customer = customer;
        this.total = total;
    }

    public double calculateDiscount() {
        return total * customer.getDiscountRate();
    }

    public Customer getCustomer(){
        return customer;
    }
}

```

Để có thể removeMiddleMan tránh việc gọi hàm MiddleMan ta có thể sửa như sau: Tạo thêm một phương thức gọi getDiscountRateOfCustomer() trong class Order. Do đó, ta có thể gọi như sau :

```
Order order = new Order (new Customer("NHT","VIP"),2.8);  
double fee = order.getDiscountRateOfCustomer();
```

```
public class Customer {  
    private String name;  
    private String membership;  
  
    public Customer(String name, String membership) {  
        this.name = name;  
        this.membership = membership;  
    }  
  
    public double getDiscountRate() {  
        return membership.equals("VIP") ? 0.2 : 0.05;  
    }  
}  
  
public class Order {  
    private Customer customer;  
    private double total;  
  
    public Order(Customer customer, double total) {  
        this.customer = customer;  
        this.total = total;  
    }  
  
    public double calculateDiscount() {  
        return total * customer.getDiscountRate();  
    }  
    public double getDiscountRateOfCustomer(){  
        return customer.getDiscountRate();  
    }  
}
```